

TD 7 : Processus, fork, signaux

IPC (进程间通信) 的通信机制, 包括共享内存 (shm)、信号量 (sem)、管道以及命名管道

Objectifs pédagogiques :

- Communications IPC : shm, sem
- Tube et Tube nommé

Donc pour P et $F2$, facile de connaitre les autres pid.

Le père P n'a qu'à stocker les pid de ses fils (obtenus par fork).

Le deuxième fils $F2$ peut hériter de son père le pid de $F1$ et utiliser e.g. `getppid()` pour trouver P .

Le premier fils $F1$, on a un souci, comment lui communiquer le pid de son petit frère ? On ne dispose pas d'API pour le retrouver (pas de `getsiblingpid...`).

Donc on utilise "pipe".

Voir aussi "man 7 pipe".

Une fois qu'on a fait "pipe", on obtient deux descripteurs de fichiers (des int), avec lesquels on va utiliser l'API standard sur fd : `read(fd, adresse, taille)`, `write(fd, adresse, taille)`, et `close(fd)`.

Pour être propre, on ferme toutes les extrémités que l'on n'utilise pas.

```

                                troisProc.cpp
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
#include <iostream>

int main3 () {
    pid_t P,F1,F2;

    P = getpid();

    int pdesc[2]; // 管道描述符数组, pdesc[0]用于读, pdesc[1]用于写
    if (pipe(pdesc) != 0) { // 创建管道, 失败时返回非0值
        perror("pipe error");
        return 1;
    }

    F1 = fork(); // 创建第一个子进程F1
    if (F1 < 0) {
        perror("fork error");
        return 1;
    } else if (F1 == 0){
        F1 = getpid();
        // F1
        close(pdesc[1]); // 子进程F1不需要写端, 关闭它 // 从管道的读端读取F2的进程ID
        if (read(pdesc[0],&F2,sizeof(pid_t)) == sizeof(pid_t)) {
            std::cout << "Je suis :" << getpid() << " P=" << P << " F1=" << F1 <<
                " F2=" << F2 << std::endl;
        } else {
            perror("erreur read");
            return 1;
        }
        close(pdesc[0]); // 读取完毕后关闭读端
        return 0;
    }

    F2 = fork();
    if (F2 < 0) {
        perror("fork error");
        return 1;
    } else if (F2 == 0){
        // F2
        F2 = getpid();
        close(pdesc[0]);
        if (write(pdesc[1],&F2,sizeof(pid_t)) == sizeof(pid_t)) {
            std::cout << "Je suis :" << getpid() << " P=" << P << " F1=" << F1 <<
                " F2=" << F2 << std::endl;
        } else {
            perror("erreur write");
            return 1;
        }
        close(pdesc[1]);
        return 0;
    }

    close(pdesc[0]);
    close(pdesc[1]);

    std::cout << "Je suis :" << getpid() << " P=" << P << " F1=" << F1 << " F2=" << F2
        << std::endl;

    wait(0);
    wait(0);

    return 0;
}

```

`int pipe(int pipefd[2]);`

`pipe()` creates a pipe, a unidirectional data channel that can be used for interprocess communication.

The array `pipefd` is used to return two file descriptors referring to the ends of the pipe.

`pipefd[0]` refers to the read end of the pipe. `pipefd[1]` refers to the write end of the pipe.

Data written to the write end of the pipe is buffered by the kernel until it is read from the read end of the pipe.

2 Tube nommé

- Pour cet exercice on a besoin des primitives suivantes (cf. man en section 2)
- `mkfifo` : crée une `fifo` (tube nommé)
 - `open/close` : ouvrir/obtenir un `filedescriptor`
 - `read/write` : avec un `filedescriptor`, un `pointeur de char`, un entier pour la taille.
 - `unlink` : efface uen entrée du système de fichier

些版本的标准库可能会忽略字符串中的空字符, 因为它们被视为任意数据

Attention si la chaine contient des `\0` les versions ici se moquent du contenu ce sont des data arbitraires, comme ce que stockent les `std::string` du c++.

On accède avec `data()` au contenu de la string (sans `\0` terminal, mais possiblement avec des `char 0` au milieu). `使用data()函数可以访问std::string的内容, 不包括字符串末尾的空字符, 但可能包括中间的空字符`

On construit la string en passant un `pointeur` et une `taille`, pas juste un `pointeur de char`, sinon il serait interprété comme une string du C et coupée sur un éventuel `\0`. `应该传递一个指针和它的大小, 而不仅仅是一个指针. 这样可以确保即使字符串中有空字符也能正确处理`

Donc vu qu'on est en flux, il est nécessaire de passer la `taille puis le contenu` en deux écritures et symétriquement en lecture. `应该先传递大小 (使用size_t类型), 然后是字符数组的实际内容`

Transmettre des chaînes de caractère = souci pour la taille de ce qui passe. On va écrire la taille de la chaîne (un `size_t`) puis son contenu au format brut tableau de char.

```

                                writer.cpp
int write (const std::string & s, int fd) {
    size_t sz = s.length(); // 获取字符串长度
    if (write (fd, &sz, sizeof(sz)) != sizeof(sz)) {
        perror("write sz"); // 首先写入字符串长度
        return -1;
    }
    // 然后写入字符串数据
    if (write (fd, s.data(), sz) != sz) {
        perror("write data");
        return -1;
    }
    std::cout << "written string len=" << sz << " : " << s << std::endl;
    return sz; // 返回写入的字符串长度
}

```

En lecture, obtenir `0` comme taille lue indique qu'on est au bout du flux/fichier. Sur un tube nommé ce n'est possible que si il n'y a aucun écrivain (aucun processus n'a de fd ouvert en écriture sur le tube). S'il reste des écrivains la lecture est bloquante.

```

                                reader.cpp
int read (std::string & s, int fd) {
    size_t sz = 0; // 首先读取字符串长度
    int rd = read (fd, &sz, sizeof(sz));
    if (rd ==0) { // 如果读取长度为0, 则表示没有更多写入的数据, 到达EOF (文件结束)
        // Ceci arrivera ssi il n'y a plus d'ecrivain => introduit la notion de EOF
        return 0;
    } else if ( rd != sizeof(sz)) {
        perror("read error"); // 如果读取长度出错, 则打印错误并返回-1
        return -1;
    }
    // 创建一个足够大的缓冲区来存储字符串数据
    char buff [sz]; // 然后读取字符串数据
    if (read (fd, buff, sz) != sz) {
        perror("read error");
        return -1;
    }
    /* ctor a deux arguments, pour éviter de couper la chaine s'il y a des \0 dedans
    */ // 从缓冲区构造字符串, 不会被内部'\0'字符截断
    s = std::string(buff, sz);
    return sz; // 返回读取的字符串长度
}

```

Les soucis à traiter :

- mkfifo + tester que ça marche + `droits` pour l'usager `S_IRUSR||S_IWUSR`.
- open => `O_RDONLY` ou `O_WRONLY` selon l'extrémité
- On accroche un "unlink"+exit au signal Ctrl-C

writer.cpp

```
void handleCtrlC (int sig) {  
    // 程序监听SIGINT信号 (通常由Ctrl+C产生), 在接收到该信号时, 它会清理管道并退出, 这样就不会留下任何痕迹。程序主要使用了mkfifo来创建命名管道, open来打开管道进行写入  
    unlink(path);  
    exit(0);  
}  
  
int main (int argc, char ** argv) {  
    if (argc < 2) {  
        // 如果没有提供管道名称, 提示用户  
        std::cerr << "Provide a name for the pipe." << std::endl;  
    }  
    // 创建命名管道, 设置适当的读写权限  
    if (mkfifo(argv[1], S_IRUSR|S_IWUSR) != 0) {  
        perror("mkfifo problem");  
        return 1;  
    }  
    // 保存管道路径  
    path = argv[1];  
    // 以只写方式打开命名管道  
    int fd = open(argv[1], O_WRONLY);  
    if (fd < 0) {  
        perror("open problem");  
        return 1;  
    }  
    // 设置信号处理函数, 以便在接收到SIGINT时调用handleCtrlC  
    signal(SIGINT, handleCtrlC);  
  
    while (true) {  
        std::string s;  
        std::cin >> s; // 从标准输入读取字符串  
        if (write(s, fd) < 0) { // 将字符串写入管道  
            break;  
        }  
    }  
  
    return 0;  
}
```

Question 3. Ecrivez un programme *reader* qui prend un chemin en argument correspondant à un tube, puis lit le texte envoyé par writer dans le tube. Le programme se termine quand il n'y a plus d'écrivain sur le tube.

Le lecteur est plus simple que writer.

reader.cpp

```
1  
2  
3 int main (int argc, char ** argv) {  
4     if (argc < 2) {  
5         std::cerr << "Provide a name for the pipe." << std::endl;  
6     }  
7  
8     int fd = open(argv[1], O_RDONLY);  
9     if (fd < 0) {  
10        perror("open problem");  
11        return 1;  
12    }  
13  
14    while (true) { // 循环读取管道中的数据  
15        std::string s;  
16        int rd = read(s, fd); // 从管道读取数据到字符串s中  
17        if (rd == 0) {  
18            break;  
19        }  
20        std::cout << s << std::endl;  
21    }  
22  
23    return 0;  
24 }
```

Question 4. Que se passe-t-il si on a plusieurs lecteurs ou plusieurs écrivains ?

多个写入者不是问题, 因为小规模地写入操作是原子的。但是, 由于我们建立的通信协议并不保证写入操作的原子性, 所以必须保证一次只进行一个写入操作。为了确保一切顺利, 需要准备一个缓冲区, 它包含了数据大小和实际的字符串

Plusieurs écrivains = aucun problème, les écritures de petite taille (BUF_SIZE vaut 4096 sur ma machine) sont atomiques. MAIS avec le protocole tel qu'il est fait, on n'écrit pas atomiquement (deux appels à write). Pour que ça se passe bien, une seule écriture est nécessaire. On pourrait assez facilement le faire en préparant un buffer qui contienne la taille ET la string.

Plusieurs lecteurs, si on a forcé une seule écriture par message, ça devrait bien se passer, mais attention, un seul lecteur peut voir les messages à la fois la lecture est destructrice.

如果强制每个消息只由单个写入, 那么通常可以保证每个读取者都能看到所有的消息。但是这种情况下的读取是破坏性的, 意味着一旦消息被一个读取者读取, 它就不再可用于其他读取者

我们不能只进行一次读取操作, 因为我们需要先读取数据大小然后再读取内容。因此, 我们应该转向一种模型, 在这种模型中, 消息有固定的大小, 这使得多读者模式成为可能, 即我们需要进行两次读取操作

De plus on ne peut pas faire un seul read, car il faut lire la taille avant de lire le contenu. Donc on devrait passer vers un modèle où les messages sont de taille fixe pour permettre le mode multi-lecteur, tel quel on est obligé de faire deux lecture.

Morale : ça marchotte, mais ce n'est pas terrible, si on a des paquets de taille fixe, c'est plus envisageable d'avoir plusieurs lecteurs.

写入者使用了一个简单的协议: 首先写入表示字符串长度的数据, 然后写入字符串本身。这样读取者就知道应该读取多少字节的数据

3 Sémaphore

P1 affiche “Ping”, et le processus P2 “Pong”

```
int main() {  
  
    // create mutex initialisé 0 // 创建一个初始化为0的信号量monsem1  
    sem_t * smutex1 = sem_open("/monsem1", O_CREAT | O_EXCL | O_RDWR , 0666, 0);  
    if (smutex1==SEM_FAILED) {  
        perror("sem create");  
        exit(1);  
    }  
  
    // create mutex initialisé 0  
    sem_t * smutex2 = sem_open("/monsem2", O_CREAT | O_EXCL | O_RDWR , 0666, 0);  
    if (smutex2==SEM_FAILED) {  
        perror("sem create");  
        exit(1);  
    }  
  
    pid_t f = fork();  
    if (f==0) {  
        // fils  
        for (int i=0; i < 10; i++) {  
            sem_wait(smutex1); // 等待smutex1  
            std::cout << "Ping" << std::endl;  
            sem_post(smutex2); // 释放smutex2  
        }  
        sem_close(smutex1);  
        sem_close(smutex2);  
    } else {  
        // pere  
  
        for (int i=0; i < 10; i++) {  
            sem_post(smutex1);  
            sem_wait(smutex2);  
            std::cout << "Pong" << std::endl;  
        }  
        sem_close(smutex1);  
        sem_close(smutex2);  
  
        // on nettoie  
        sem_unlink("/monsem1");  
        sem_unlink("/monsem2");  
        wait(nullptr);  
    }  
    return 0;  
}
```

Question 2. On considère à présent N processus qui doivent s'alterner (circulairement). Combien de sémaphores faut-il introduire ? Proposez une réalisation de ce comportement.

N 个进程需要轮流执行，每个进程都在等待一个信号量，执行一段操作（例如打印“Ping”），然后释放下一个信号量，以此类推，形成一个循环。

```
#define N 4  
  
int main() {  
  
    sem_t * mutex [N]; // 定义信号量数组  
  
    for (int i=0; i < N ; i++) {  
        // create mutex initialisé 0 sauf le premier  
        int semval = 0;  
        if (i==0) { // 第一个信号量初始化为1, 其余初始化为0  
            semval = 1;  
        }  
        std::string semname = "/monsem"+std::to_string(i);  
        mutex[i] = sem_open(semname.c_str(), O_CREAT | O_EXCL | O_RDWR , 0666, semval);  
        if (mutex[i]==SEM_FAILED) {  
            perror("sem create");  
            exit(1);  
        }  
    }  
  
    for (int i=0; i < N ; i++) {  
        pid_t f = fork();  
        if (f==0) {  
            // fils  
            for (int round=0; round < 10; round++) {  
                sem_wait(mutex[i]);  
                std::cout << "Ping" << i << std::endl;  
                sem_post(mutex[(i+1)%N]);  
            }  
            return 0;  
        }  
    }  
  
    for (int i=0; i < N ; i++) {  
        wait(nullptr);  
    }  
  
    // on nettoie  
    for (int i=0; i < N ; i++) {  
        sem_unlink( ("/monsem"+std::to_string(i)).c_str() );  
    }  
    return 0;  
}
```

4 Mémoire partagée

Question 1. En se limitant aux seuls sémaphores, modifier le Stack pour que : **pop bloque si vide**, **push bloque si plein**, et que les données partagées *sz* et *tab* soit protégées des accès concurrents. On suppose que le Stack est alloué dans un segment de mémoire partagée, et qu'il sera partagé entre processus.

On note au passage les différences avec l'exo précédent: on y a utilisé *sem_open*, *sem_unlink*. Nos sémaphores sont ici "anonymes" au sens où ils n'ont pas d'entrée spécifique (inode) dans le système de fichier.

Donc on a besoin de trois sémaphores :

保护对sz和tab的访问。每次进入和退出关键区域（即修改栈时）都会执行P（等待）和V（释放）操作

- un sem qui joue le rôle de **mutex**, il protège *sz* et *tab*. Initialisé à **1**, les accès aux attributs sont des sections critiques, chacun fait un *P* avant et un *V* après avoir lu/écrit *tab* ou *sz*.
- deux sem qui jouent le rôle de mécanisme de notification et de contrôle **anti débordement**.
防止栈溢出的信号量
 - sempop** : initialisé à **0**, on *P* dessus avant de *pop*, on *V* dessus après un *push*. Donc au départ 0, c'est vide, la valeur du compteur reflète *size*.
 - sempush** : initialisé à **MAX**, on *P* dessus avant de *push*, on *V* dessus après un *pop*. Donc au départ MAX, c'est vide on peut faire MAX *push*. La valeur du compteur reflète $MAX - size$.

```

#ifndef __STACK_HH__
#define __STACK_HH__

#include <cstring> // size_t
#include <semaphore.h>
#include <iostream>

namespace pr {

#define STACKSIZE 100

// T = un type numerique (contrat de valarray)
template<typename T>
class Stack {
    T tab [STACKSIZE]; // 数组, 用于存储栈中的元素
    size_t sz; // 栈当前元素的数量
    sem_t mutex; // 用于互斥的信号量
    sem_t sempop; // 用于控制pop操作的信号量
    sem_t sempush; // 用于控制push操作的信号量

public:
    // 构造函数
    Stack () : sz(0) {
        sem_init(&mutex, 1, 1); // 初始化互斥信号量, 初始值为1
        sem_init(&sempop, 1, 0);
        sem_init(&sempush, 1, STACKSIZE);
        memset(tab, 0, sizeof tab); // 将栈内存区域清零
    }

    ~Stack() {
        sem_destroy(&mutex); // 销毁互斥信号量
        sem_destroy(&sempop);
        sem_destroy(&sempush);
    }

    T pop () {
        sem_wait(&sempop); // 等待直到栈非空
        sem_wait(&mutex); // 进入互斥区域
        T toret = tab[--sz]; // 取出栈顶元素
        sem_post(&mutex); // 离开互斥区域
        sem_post(&sempush); // 通知可以进行push操作
        return toret; // 返回栈顶元素
    }

    void push(T elt) {
        sem_wait(&sempush); // 等待直到栈未满
        sem_wait(&mutex);
        tab[sz++] = elt; // 将元素压入栈
        sem_post(&mutex);
        sem_post(&sempop);
    }
};

#endif

```

每次尝试进行pop操作时, 都会减少sempop的值(等待操作)。如果栈为空(即sempop的值为0), pop操作将会阻塞, 因为信号量不允许负值。这意味着pop操作必须等待直到有元素被push进栈。

执行V操作在push之后: 每当一个元素被推入(push)栈时, 都会增加sempop的值(释放操作), 这允许正在阻塞的pop操作继续执行。
计数器的值反映了栈中元素的数量: sempop的值表示栈中当前可供弹出的元素数量。

int sem_init(sem_t *sem, int pshared, unsigned int value);
pshared: 如果这个值非零, 那么信号量在多个进程间是共享的; 如果这个值为零, 信号量只能被初始化它的进程的线程所共享
Attention à bien initialiser dans le constructeur, et aussi à les libérer à la destruction. On veut s'en servir entre processus (et non entre thread) du coup on passe 1 en deuxième argument au au sem_init.

TME 7 : Mémoire Partagée, Sémaphores

1 Fork, exec, pipe

Question 1. On souhaite simuler le comportement d'un shell pour chaîner deux commandes : la sortie de la première commande doit alimenter l'entrée de la deuxième commande.

Ecrire un programme "pipe" qui a ce comportement. On utilisera :

- **fork et exec**, on suggère d'utiliser la version **execv** (**execvp** etc), pour utiliser le processus courant.
- **pipe** pour créer un tube, **write** pour écrire dans le tube, **read** pour lire dans le tube.
- **dup2(fdl, fd2)** pour remplacer fd2 par fdl, de façon à substituer aux entrées sorties "normales" les extrémités du pipe. **dup2** est utilisé pour dupliquer un fichier descripteur, en remplaçant le fichier descripteur d'origine par le fichier descripteur dupliqué.

On pourra tester avec par exemple (le backslash protège l'interprétation du pipe par le shell) :

```
pipe /bin/cat pipe.cpp | /bin/wc -l
```

NB 1 : Il faut itérer sur les arguments *argc, argv* passés au main pour chercher le symbole `pipe`, et construire deux tableaux d'arguments (des `const char*`) terminés par `nullptr` pour invoquer `execv`.
应该迭代main函数传递的argc和argv参数来构造两个参数数组 (以nullptr结尾的const char*数组), 以便调用execv

```

pipe.cpp

// 执行命令的函数, 使用execv系统调用
void myexec (int argc, char ** args) {
    std::cerr << "args : " ;
    for (int i = 0; i < argc; i++) {
        if (args[i] != nullptr) {
            std::cerr << args[i] << " ";
        } else {
            break;
        }
    }
    std::cerr << std::endl;
    // 执行命令, 替换当前进程
    if (execv (args[0], args) == -1) {
        perror ("exec");
        exit (3);
    }
}

int main (int argc, char ** argv) {
    // On partitionne les args en deux jeux d'arguments
    char * args1[argc]; // 将命令行参数分割成两组命令
    char * args2[argc];

    // 初始化参数数组
    memset(args1, 0, argc * sizeof(char*));
    memset(args2, 0, argc * sizeof(char*));

    int arg = 1;
    for ( ; arg < argc; arg++) {
        if (! strcmp(argv[arg], "|")) { // 如果当前参数不是管道符号
            arg++;
            break;
        } else {
            args1[arg-1] = argv[arg]; // 加入第一组命令
        }
    }

    for (int i = 0; arg < argc; i++, arg++) {
        args2[i] = argv[arg]; // 剩下的参数属于第二组命令
    }

    // OK, args1 = paramètre commande 1, args2 = deuxième commande.

    int tubeDesc[2]; // 管道描述符数组
    pid_t pid_fils;
    if (pipe (tubeDesc) == -1) {
        perror ("pipe");
        exit (1);
    }

    if ( (pid_fils = fork ( )) == -1) {
        perror ("fork");
        exit (2);
    }

    if (pid_fils == 0) { /* fils 1 */
        // ...
    }
}

```

```

63 dup2(tubeDesc[1], STDOUT_FILENO); // 将标准输出重定向到管道的写端
64 close (tubeDesc[1]);
65 close (tubeDesc[0]);
66
67 myexec(argc, args1); // 执行第一组命令
68 // on ne revient pas du exec
69
70
71 // pere donc ici
72 if ( (pid_fils = fork ( )) == -1) {
73     perror ("fork");
74     exit (2);
75 }
76 if (pid_fils == 0) { /* fils 2 */
77     dup2(tubeDesc[0], STDIN_FILENO);
78     close (tubeDesc[0]); // 将标准输出重定向到管道的读端
79     close (tubeDesc[1]);
80
81     myexec(argc, args2);
82 }
83
84 // important
85 close (tubeDesc[0]);
86 close (tubeDesc[1]);
87
88 wait(0);
89 wait(0);
90 return 0;
91 }

```



```

void producteur (Stack<char> * stack) {
    char c ;
    while (cin >> c) { // 循环读取字符直到输入结束
        stack->push(c); // 将字符推入栈中
    }
}

void consommateur (Stack<char> * stack) {
    while (true) {
        char c = stack->pop(); // 从栈中弹出字符
        cout << c << std::flush // 输出字符并刷新输出, 确保立即显示
    }
}

std::vector<pid_t> tokill; // 用于存储需要杀死的进程的ID

void killem (int) {
    for (auto p : tokill) { // 遍历tokill数组中的每个进程ID
        kill(p, SIGINT); // 向每个进程发送中断信号
    }
}

int main () {
    size_t shmsize = sizeof(Stack<char>); // 计算栈对象的大小
    std::cout << "Allocating segment size " << shmsize << std::endl;

    int fd;
    void * addr; // 指向共享内存段的指针

    bool useAnonymous = false; // 标志位, 决定使用匿名内存还是命名共享内存
    if (useAnonymous) { // 如果使用匿名内存 // 创建匿名内存映射
        addr = mmap(nullptr, shmsize, PROT_READ | PROT_WRITE, MAP_SHARED |
            MAP_ANONYMOUS, -1, 0);
        if (addr == MAP_FAILED) {
            perror("mmap anonymous");
            exit(1);
        }
    } else { // 否则使用命名共享内存 // 创建共享内存对象
        fd = shm_open("/myshm", O_CREAT | O_EXCL | O_RDWR, 0666);
        if (fd < 0) {
            perror("shm_open");
            return 1;
        }
        // 调整共享内存对象的大小
        if (ftruncate(fd, shmsize) != 0) {
            perror("ftruncate");
            return 1;
        }
    }

    // 映射共享内存段
    addr = mmap(nullptr, shmsize, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    if (addr == MAP_FAILED) {
        perror("mmap anonymous");
        exit(1);
    }

    // 使用"定位new"在共享内存中构造栈对象
    // In place new : faire le new à l'adresse fournie, supposée assez grande.
    Stack<char> * s = new (addr) Stack<char>();

    // 创建生产者进程
    pid_t pp = fork();
    if (pp==0) { // 调用生产者函数
        producteur(s);
        return 0;
    }

    pid_t pc = fork();
    if (pc==0) {
        consommateur(s);
        return 0;
    } else {
        tokill.push_back(pc);
    }
    // 将消费者进程ID加入tokill数组

    signal(SIGINT, killem);
    // 设置信号处理函数, 用于在接收到SIGINT信号时杀死子进程
    wait(0);
    wait(0);

    // invoque le destructeur, mais pas delete
    s->~Stack(); // 调用栈对象的析构函数, 但不释放内存, 因为它是在共享内存中
    if (munmap(addr, shmsize) != 0) {
        perror("munmap"); // 解除内存映射
        exit(1);
    }
    if (! useAnonymous) { // 如果使用命名共享内存, 需要解除链接
        if (shm_unlink("/myshm") != 0) {
            perror("sem unlink");
        }
    }
}

```

Les processus consommateur et producteur ne peuvent pas communiquer à travers le stack actuellement, car il n'est pas dans une zone partagée. Proposez une correction du code pour que le Stack soit dans un segment de mémoire partagée anonyme. Comparez le code à une version dans un segment nommé.

Q3 Le programme ne se termine pas actuellement, ajoutez une gestion de signal pour que le main interrompe le(s) consommateur(s) si on lui envoie un Ctrl-C (SIGINT).

```

// 映射共享内存段
addr = mmap(nullptr, shmsize, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
if (addr == MAP_FAILED) {
    perror("mmap anonymous");
    exit(1);
}

// 使用"定位new"在共享内存中构造栈对象
// In place new : faire le new à l'adresse fournie, supposée assez grande.
Stack<char> * s = new (addr) Stack<char>();

// 创建生产者进程
pid_t pp = fork();
if (pp==0) { // 调用生产者函数
    producteur(s);
    return 0;
}

pid_t pc = fork();
if (pc==0) {
    consommateur(s);
    return 0;
} else {
    tokill.push_back(pc);
}
// 将消费者进程ID加入tokill数组

signal(SIGINT, killem);
// 设置信号处理函数, 用于在接收到SIGINT信号时杀死子进程
wait(0);
wait(0);

// invoque le destructeur, mais pas delete
s->~Stack(); // 调用栈对象的析构函数, 但不释放内存, 因为它是在共享内存中
if (munmap(addr, shmsize) != 0) {
    perror("munmap"); // 解除内存映射
    exit(1);
}
if (! useAnonymous) { // 如果使用命名共享内存, 需要解除链接
    if (shm_unlink("/myshm") != 0) {
        perror("sem unlink");
    }
}
}

```

3 Messagerie par mémoire partagée

```

#define MAX_MESS 50
#define MAX_USERS 10
#define TAILLE_MESS 10

struct message {
    long type; // 消息类型
    char content[TAILLE_MESS]; // 消息内容
};

struct myshm {
    // 服务器已读取的消息数, 已转发
    int read; /* nombre de messages retransmis par le serveur */
    int write; /* nombre de messages non encore retransmis par le serveur */
    int nb; /* nombre total de messages emis */
    sem_t sem;
    struct message messages[MAX_MESS]; // 消息数组
};

char *getName(char *name);

#endif

```

common.h

```

1 #include <chat_common.h>
2
3 int shouldexit = 0; // 控制退出的标志变量, 初始化为0
4
5 void exithandler(int sig){
6     shouldexit = sig; /* i.e !=0 */
7 } // 退出处理函数, 将信号值赋给shouldexit变量
8
9 struct user {
10     char *name;
11     struct myshm *shm; // 指向共享内存结构的指针
12
13 } *users[MAX_USERS];
14 // 定义用户数组, 最大用户数为MAX_USERS
15
16 int main(int argc, char *argv[]){
17     char *shmname; // 共享内存名称指针
18     int shm_id, i; // 共享内存ID和循环变量
19     struct myshm *shm_pere; // 父共享内存结构指针
20     struct sigaction action; // 信号动作结构体
21
22     if (argc <= 1) { // 如果参数不足
23         fprintf(stderr, "Usage: %s id_server\n", argv[0]);
24         exit(EXIT_FAILURE);
25     }
26
27     /* Met un handler pour arrêter le programme de façon clean avec Ctrl-C */
28     action.sa_handler = exithandler; // 设置信号处理函数
29     action.sa_flags = 0; // 设置信号处理标志
30     sigemptyset(&action.sa_mask); // 初始化信号集为空
31     sigaction(SIGINT, &action, 0); // 设置对SIGINT信号的处理动作
32
33     /* 创建共享内存标识符 */
34     /* On crée l'identifiant "/>

```

```

70 /* On récupère le premier message et on incrémente le nombre lu */
71 struct message message = shm_pere->messages[shm_pere->read];
72 // 从共享内存中的消息队列里读取位置 read 指向的消息, 并将其存储到局部变量 message 中
73
74 switch(message.type){
75     case 0:{
76         /* Connexion */
77         char *username; // 用户名指针
78         int shm_id_user; // 用户共享内存ID
79         i=0; // 索引变量初始化
80
81         /* Récupère la première case user vide */
82         while(i<MAX_USERS && users[i] != NULL) i++; // 查找第一个空的用户槽位 */
83         if(i == MAX_USERS){perror("impossible d'ajouter l'utilisateur"); exit(EXIT_FAILURE);}
84
85         users[i] = malloc(sizeof(struct user)); // 为新用户分配内存
86         if(users[i] == NULL){perror("impossible d'ajouter l'utilisateur"); exit(EXIT_FAILURE);}
87
88         /* Copie le nom du fils */ /* 复制文件名到用户结构 */
89         users[i]->name = malloc((strlen(message.content) + 1) * sizeof(char));
90         strcpy(users[i]->name, message.content);
91         users[i]->name[strlen(message.content)] = '\0'; // 确保字符串以NULL结尾
92
93         printf("Connexion de %s\n", users[i]->name);
94
95         /* Récupère le segment partagé de l'utilisateur et le mappe */
96         username = message.content; // 获取用户名
97         if((shm_id_user = shm_open(username, O_RDWR | O_CREAT, 0666)) == -1) {
98             perror("shm_open shm_fils");
99             exit(EXIT_FAILURE);
100         }
101         if((users[i]->shm = mmap(NULL, sizeof(struct myshm), PROT_READ | PROT_WRITE,
102             MAP_SHARED, shm_id_user, 0)) == MAP_FAILED){
103             perror("mmap shm_fils");
104             exit(EXIT_FAILURE);
105         }
106
107         break;
108     }
109     case 1:{
110         /* Envoi de message */
111         int temp = 0; // 用于计数的临时变量
112         printf("Réception message serveur : %s\n", message.content);
113         for(i=0; i<MAX_USERS; i++){
114             if(users[i] != NULL){
115                 struct message msg; // 创建消息结构体实例
116                 struct myshm *shm = users[i]->shm; // 获取用户的共享内存指针
117                 temp++;
118
119                 /* 复制消息到子进程的消息队列 */
120                 /* On copie le message dans la file du fils */
121                 msg.type = 1; // 设置消息类型
122                 strcpy(msg.content, message.content); // 复制消息内容
123
124                 sem_wait(&(shm->sem)); // 等待信号量
125
126                 while(shm->nb == MAX_MESS){
127                     sem_post(&(shm->sem)); // 如果消息队列已满, 释放信号量, 稍后重试
128                     sleep(1);
129                     sem_wait(&(shm->sem)); // 再次等待信号量
130
131                     /* TODO : vérifier que c'est pas plein */
132                 }
133
134                 shm->messages[shm->write] = msg; // 将消息放入共享内存的消息队列中
135                 shm->write = (shm->write + 1) % MAX_MESS; // 循环队列, 更新写指针位置
136                 shm->nb++; // 增加消息数量
137
138                 sem_post(&(shm->sem)); // 释放信号量, 允许其他进程/线程访问共享内存
139             }
140         }
141
142         if(temp == 0){ // 如果没有找到用户, 打印信息
143             printf("Nobody found!\n");
144         }
145         break;
146     }
147     case 2:{
148         printf("Déconnexion de %s\n", message.content);
149         /* Déconnexion */
150         i=0; // 初始化用户索引
151
152         /* On récupère le user correspondant à l'utilisateur qui se déconnecte */
153         while(i<MAX_USERS && (users[i] == NULL || strcmp(users[i]->name, message.content)
154             != 0)) i++; // 寻找要断开连接的用户 找到一个非 NULL 的用户结构体, 其名字与消息内容中提供的名字完全一致。当找到这样的用户时, 表示这个用户是想断开连接的用户, 循环终止
155         if(i == MAX_USERS){
156             perror("Something went wrong! L'utilisateur n'existe pas!");
157             exit(EXIT_FAILURE);
158         }
159
160         // 释放用户占用的内存资源
161         /* On libère l'espace mémoire */
162         free(users[i]->name); // 释放用户名字符串
163         munmap(users[i]->shm, sizeof(struct myshm)); // 取消映射共享内存
164         free(users[i]); // 释放用户结构体内存
165         users[i] = NULL; // 将用户指针设置为NULL
166
167         break;
168     }
169     // 更新共享内存的读指针位置, 并减少消息数量
170     shm_pere->read = (shm_pere->read + 1) % MAX_MESS;
171     shm_pere->nb--;
172 }
173
174 // 关闭信号量, 取消共享内存映射, 解除共享内存的连接, 并返回成功退出状态
175 sem_close(&(shm_pere->sem));
176
177 munmap(shm_pere, sizeof(struct myshm));
178
179 shm_unlink(shmname);
180
181 return EXIT_SUCCESS;
182 }

```

```

1 #include <chat-common.h>
2
3 int shouldexit = 0; // 控制是否退出程序的全局变量
4
5 struct myshm *shm_client, *shm_pere; // 定义指向共享内存结构的指针
6 // 读线程函数
7 void *reader(void* arg){
8     while(!shouldexit) // 当没有接收到退出信号时持续运行
9         sem_wait(&(shm_client->sem)); // 等待信号量, 以进入临界区
10
11     /* Si il y a eu au moins une écriture */ // 检查是否有消息需要读取
12     if(shm_client->read != shm_client->write || shm_client->nb == MAX_MESS){
13         /* On récupère le premier message et on incrémente le nombre lu */
14         struct message message = shm_client->messages[shm_client->read];
15         shm_client->read = (shm_client->read + 1) % MAX_MESS; // 读取消息并更新读索引
16         printf("%s", message.content);
17         shm_client->nb--; // 减少未读消息的数量
18     }
19     sem_post(&(shm_client->sem));
20 }
21 pthread_exit(NULL); // 退出线程
22 return NULL;
23 }
24
25 // 写线程函数
26 void *writer(void* arg){
27     while(!shouldexit) // 当没有接收到退出信号时持续运行
28         struct message msg; // 定义消息结构体
29
30     msg.type = 1; // 设置消息类型
31     fgets(msg.content, TAILLE_MESS, stdin); // 从标准输入读取消息内容
32
33     /* Evite d'envoyer un message avec Ctrl-C dedans */
34     if(shouldexit) break; // 检查退出条件
35
36     sem_wait(&(shm_pere->sem));
37
38     // 检查共享内存是否满了
39     while(shm_pere->nb == MAX_MESS){
40         sem_post(&(shm_pere->sem));
41         sleep(1);
42         sem_wait(&(shm_pere->sem));
43     }
44
45     // 写入消息到共享内存并更新写索引
46     shm_pere->messages[shm_pere->write] = msg;
47     shm_pere->write = (shm_pere->write + 1) % MAX_MESS;
48     shm_pere->nb++; // 增加消息的数量
49
50     sem_post(&(shm_pere->sem));
51 }
52 pthread_exit(NULL);
53 return NULL;
54 }
55
56 void exithandler(int sig){
57     shouldexit = sig; // i.e. !=0 */
58 } // 接收到信号后, 设置退出标志
59
60 int main(int argc, char *argv[]){
61     char *shm_client_name, *shm_pere_name; // 客户端和服务器的共享内存名称
62     int shm_client_id, shm_pere_id;

```

```

60 pthread_t tids[2]; // 线程ID数组
61 struct message msg; // 消息结构体
62 struct sigaction action;
63
64 if (argc <= 2) {
65     fprintf(stderr, "Usage: %s id_client id_server\n", argv[0]);
66     exit(EXIT_FAILURE);
67 }
68
69 printf("Connexion à %s sous l'identité %s\n", argv[2], argv[1]);
70
71 /* Met un handler pour arrêter le programme de façon clean avec Ctrl-C */
72 action.sa_handler = exithandler; // 信号处理函数
73 action.sa_flags = 0;
74 sigemptyset(&action.sa_mask); // 初始化信号集
75 sigaction(SIGINT, &action, 0); // 设置信号
76
77 shm_client_name = (argv[1]); // 客户端共享内存名称
78 shm_pere_name = (argv[2]); // 服务器共享内存名称
79
80 /* Crée le segment en lecture écriture */
81 if ((shm_client_id = shm_open(shm_client_name, O_RDWR | O_CREAT, 0666)) == -1) {
82     perror("shm_open shm_client");
83     exit(errno);
84 }
85 if ((shm_pere_id = shm_open(shm_pere_name, O_RDWR | O_CREAT, 0666)) == -1) {
86     perror("shm_open shm_pere");
87     exit(errno);
88 }
89
90 // 分配共享内存段大小
91 /* Allouer au segment une taille de NB_MESS messages */
92 if (ftruncate(shm_client_id, sizeof(struct myshm)) == -1) {
93     perror("ftruncate shm_client");
94     exit(errno);
95 }
96
97 /* Mappe le segment en read-write partagé */ // 将共享内存段映射到进程的地址空间
98 if ((shm_pere = mmap(NULL, sizeof(struct myshm), PROT_READ | PROT_WRITE, MAP_SHARED,
99     shm_pere_id, 0)) == MAP_FAILED){
100     perror("mmap shm_pere");
101     exit(errno);
102 }
103 if ((shm_client = mmap(NULL, sizeof(struct myshm), PROT_READ | PROT_WRITE, MAP_SHARED,
104     shm_client_id, 0)) == MAP_FAILED){
105     perror("mmap shm_pere");
106     exit(errno);
107 }
108
109 // 初始化客户端共享内存结构
110 shm_client->read = 0; // 已读消息计数器
111 shm_client->write = 0; // 待写消息计数器
112 shm_client->nb = 0; // 消息数量
113
114 /* Initialisation du sémaphore (mutex) */
115 if (sem_init(&(shm_client->sem), 1, 1) == -1){
116     perror("sem_init shm_client");
117     exit(errno);
118 }
119
120 // 创建线程并等待它们结束
121 msg.type = 0;
122 strcpy(msg.content, argv[1]); // 复制命令行第一个参数作为消息内容

```

```

123 // 等待父共享内存的信号量
124 sem_wait(&(shm_pere->sem));
125
126 // 将消息写入父共享内存的消息队列中
127 shm_pere->messages[shm_pere->write] = msg;
128 shm_pere->write = (shm_pere->write + 1) % MAX_MESS;
129 shm_pere->nb++; // 增加消息数量
130
131 // 释放父共享内存的信号量, 允许其他进程操作共享内存
132 sem_post(&(shm_pere->sem));
133
134 /* On crée les threads et on attend qu'ils se terminent */
135 pthread_create(&(tids[0]), NULL, reader, NULL); // 创建读线程
136 pthread_create(&(tids[1]), NULL, writer, NULL); // 创建写线程
137 pthread_join(tids[0], NULL); // 等待读线程结束
138 pthread_join(tids[1], NULL); // 等待写线程结束
139
140 printf("Déconnexion...\n");
141
142 msg.type = 2; // 设置消息类型为断开连接
143 sem_wait(&(shm_pere->sem));
144
145 // 再次将消息写入父共享内存的消息队列
146 shm_pere->messages[shm_pere->write] = msg; // 更新写索引
147 shm_pere->write = (shm_pere->write + 1) % MAX_MESS;
148 shm_pere->nb++;
149
150 sem_post(&(shm_pere->sem));
151
152 sem_close(&(shm_client->sem));
153
154 munmap(shm_client, sizeof(struct myshm));
155
156 shm_unlink(shm_client_name);
157
158 return EXIT_SUCCESS;
159 }

```

